

The sharp-field report, explained for an intro-ML reader

A plain-language companion to [MF_Sharp_HighFreq_Report.pdf](#)

Prepared 2026-07-02 · same audited facts and numbers as the technical report — nothing new is claimed here, everything is just explained more slowly, with analogies to things an intro machine-learning class already covers.

How to read this document. It assumes one intro ML course — you know what train/test splits, overfitting, MSE loss, and a CNN are — but nothing about PDEs, Fourier operators, or multi-fidelity modeling. Every project-specific idea is built up before it is used. Boxes like this one translate a technical concept into something you have already seen. For the from-scratch tour of the model zoo itself (what an FNO is, what FiLM is, all 30 families), see the companion page [MFFP_Model_Explanations_Beginner.html](#) — this document assumes you have that vocabulary and focuses only on the new material in the sharp-field report.

0. A 90-second refresher

Skip this if you've already read the companion model-zoo page.

The project predicts a **high-fidelity (HF)** field — think of it as a fine-grained image of some physical quantity, like temperature or pressure over a 2-D surface — from a **low-fidelity (LF)** version of the same thing that is cheap to produce but blurrier and less accurate, plus a handful of numbers describing the scenario. LF is abundant (you can generate thousands of cheap examples); HF is scarce (each one is expensive), so the model has to learn to combine "lots of cheap and rough" with "a little expensive and precise."

The workhorse architecture is a **Fourier Neural Operator (FNO)**. Instead of convolving with small local filters like a normal CNN, its core layer takes the 2-D Fourier transform of the image, keeps only a small number of low frequencies (called **modes**), multiplies them by learned weights, and transforms back. Keeping only the lowest 12 modes per axis (a couple of families keep 16 or 20) is like describing an image using only its dozen broadest brushstrokes — extremely efficient for smooth images, and, as this report explains, a hard ceiling on how sharp an image the network can ever produce.

Most families in the zoo either (a) pretrain on lots of LF data and fine-tune on scarce HF data (LF only ever shapes the *weights*, never enters a prediction directly), or (b) predict $\text{HF} = \text{a baseline} + \text{a learned correction}$, where the baseline is often the LF field itself. The headline finding so far, on the original 17 mostly-smooth datasets, is that the simplest version of (a) — plain FNO pretrain-then-finetune with **FiLM** conditioning (a small side-network that rescales and shifts each layer's activations based on the scenario) — beats every fancier fusion mechanism in the zoo.

The new SURF-2026 datasets are specifically designed to ask: does that headline result survive when the field has *sharp* features — shockwaves, phase-transition interfaces — instead of smooth ones? This

report's answer is: not without changes, and it explains exactly why, then proposes nine new model families to fix it.

Executive summary, in plain language

The question. Everything the model zoo has learned about "what wins" was learned on smooth data. Three new datasets — 2-D compressible-flow shockwaves (Euler equations), Cahn–Hilliard phase-separation interfaces, and Kuramoto–Sivashinsky (a smooth, chaotic control case) — test whether the winning recipe survives contact with sharp, high-frequency content.

Finding 1 — there are three separate reasons to expect the current zoo to struggle, and you can see all three without training a single model.

If you know intro ML, think: this is like knowing in advance that a model trained only on 32×32 downsampled images cannot classify fine texture, no matter how long you train it — the information was thrown away before training started. Two of the three walls below are exactly that kind of argument; the third is closer to a classic bias-variance point about what squared-error loss actually optimizes for.

Wall 1 — the frequency budget is too small for sharp edges (the "spectral wall"). Every FNO family keeps only 12–20 of the lowest frequencies. For smooth fields that's nearly free — almost all of the useful information is in those low frequencies anyway. A sharp edge (a shockwave, a phase boundary) is different: mathematically, its frequency content decays very slowly as you go up in frequency, so a lot of the "edge information" lives in exactly the high frequencies the model throws away. No amount of training changes this — it is a hard floor on accuracy, roughly 3–10% relative error on shock fields, set by the preprocessing choice of "keep only 12 modes," not by the model's skill.

Wall 2 — "predict the error" breaks when the cheap version puts the edge in the wrong place. Many families predict $HF = LF + \text{correction}$, the same idea as gradient boosting: fit a first model, then fit a second, smaller model to clean up its mistakes. That only works if the mistakes are small nudges. Here, the cheap (LF) solver doesn't just blur a shockwave — on a coarser grid it also **moves** it a few pixels. So the "correction" a second model must learn is not a small nudge: it has to erase a shockwave in one place and draw a new one somewhere else. That is a *harder* problem than just predicting the shockwave's position correctly the first time — the opposite of what a correction step is supposed to buy you.

Wall 3 — the loss function and the scoring metric both quietly prefer a blurry answer over a sharp-but-slightly-wrong one. This is the most subtle wall, and the most important to internalize. Every family trains with mean-squared-error (MSE), and every leaderboard score (relative-L2 error, the Elo ranking, the composite score used to pick a winner) is built on the same squared-distance idea. Squared error is minimized, on average, by predicting the *conditional mean* of all plausible answers. If a model can't be 100% sure exactly where a shockwave sits, MSE tells it to output something between all the plausible positions — a smooth ramp, not a sharp jump. That smeared answer is not a training failure; it is *the mathematically optimal thing to output under this loss*. Worse, the scoring metric agrees: a blurred ramp scores a lower (better) relative-L2 error than a razor-sharp shockwave placed a few pixels off. So a model

that "chickens out" and blurs is rewarded, and a model that commits to a sharp, confident, slightly-off answer is punished — by the exact metric used to rank the leaderboard.

Why this matters beyond this project: this is a completely general fact about squared-error training and squared-error scoring, not something specific to FNOs. Any time a model has genuine uncertainty about *where* something is (an edge, a boundary, an object), MSE-trained regression will blur that uncertainty into the output. Classification doesn't have this problem — a softmax can be "40% sure it's a 3, 60% sure it's an 8" without ever producing a blurry hybrid digit — but pixel-wise regression can't express "I'm not sure where the edge is" any other way than smearing it.

Finding 2 — this isn't speculation; other benchmarks have already measured exactly this failure mode. A public benchmark called The Well ran an FNO on the same *family* of shockwave problems the new dataset uses, and found the FNO about **2.7× worse** than the best convolutional model tested, a ConvNeXt-U-Net (0.408 vs. 0.153 on their error metric; an ordinary U-Net scored 0.183, still far ahead of the FNO). Two other benchmarks (APEBench, PDEBench) report the same pattern: spectral models like FNO win when a system's energy concentrates in a few frequencies, and lose once the spectrum fills up across all frequencies — which is exactly what a shockwave does. Expectation: the current leaderboard order should roughly survive on the smooth control dataset (Kuramoto–Sivashinsky) and flip on the shockwave and interface datasets, unless the zoo changes.

Finding 3 — the current winning models throw away exactly the information that would help most here. The winning "pretrain-then-finetune" families never look at the actual LF *image* when making a prediction — LF only shaped the weights during pretraining, then is discarded. On smooth data that's fine, because the handful of scenario numbers is enough to pin down a smooth solution. On sharp data, the LF image is where the *shape and rough location* of the edge lives — throwing it away at prediction time throws away the one thing that would tell the model roughly where to put the shockwave. Four completely unrelated fields — weather forecasting, seismic imaging, statistics, and transonic aerodynamics — independently discovered the same fix decades ago: **before you compare or correct two versions of an image, first figure out how to shift/warp one onto the other, and only then correct the remaining amplitude difference.** No published multi-fidelity *operator-learning* method (the kind of model this project builds) does that yet — related ideas exist elsewhere in multi-fidelity modeling more broadly, but not this specific mechanism, in this specific setting. It's the single clearest opportunity for a genuinely new model family (proposal P3 below).

The nine proposals, in the order to build them. Two quick, low-risk experiments come first and answer "is the problem the model's frequency budget, or the training loss?" separately (P1, P2). Then the report proposes the "align, then correct" idea as a flagship new family (P3), plus two more multi-fidelity-specific mechanisms (P4, P5). The last four are dataset specialists and bigger bets (P6–P9).

Part 0 — The three walls, in detail

0.1 The spectral wall: not enough frequencies for an edge

Figure 1 — The spectral wall: a 12-mode spectral backbone cannot represent sharp fields, no matter how it is trained

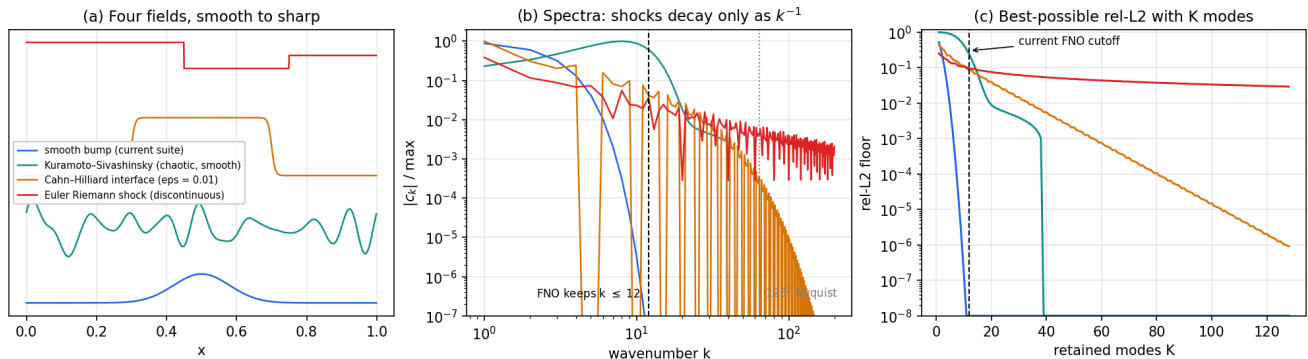


Figure 1 — Reconstructing a smooth field from only 12 low frequencies loses almost nothing; reconstructing a sharp edge from the same 12 frequencies leaves a large, un-trainable error. The gap between the two curves is a hard ceiling, not a training problem.

Think of the Fourier transform as rewriting an image as a sum of sine-wave patterns at different frequencies: a few "broad stroke" low frequencies, and many "fine detail" high frequencies. An FNO keeps only the lowest 12 (or, for two families, 16 or 20) of these per axis and throws the rest away — a fixed, un-trainable preprocessing choice built into the architecture, not something more training data can fix.

For a smooth field, this is a great trade: the broad strokes really do capture almost everything, so 12 frequencies reconstruct the field almost perfectly. For a sharp edge — a shockwave, a phase boundary — the math works against you: a true discontinuity's frequency content falls off very slowly as frequency increases (it decays as $1/k$, where k is the frequency index), so a meaningful fraction of "what the edge looks like" is smeared across *all* frequencies, including the ones being discarded. The result is a floor of roughly 3–10% relative error on shockwave fields that exists purely because of the 12-mode truncation — before any weight has even been trained. The Cahn–Hilliard interface sits in between: it's smooth once you zoom in past its width, but that width can be thinner than what 12 low frequencies can resolve, so the same floor applies whenever the dataset's grid doesn't give the interface enough "room" to look smooth.

If you know intro ML, think: this is exactly like training an image classifier only on images that have been passed through a fixed 12×12-pixel box blur. No architecture change or extra epoch of training recovers detail that the preprocessing already destroyed — you have to change the preprocessing (here: keep more frequencies, or stop relying on frequencies alone).

0.2 Misalignment breaks the "predict the error" trick

Figure 2 — Why 'HF = LF + delta' breaks at shocks: when LF mispositions the discontinuity, the residual must cancel one shock and build another

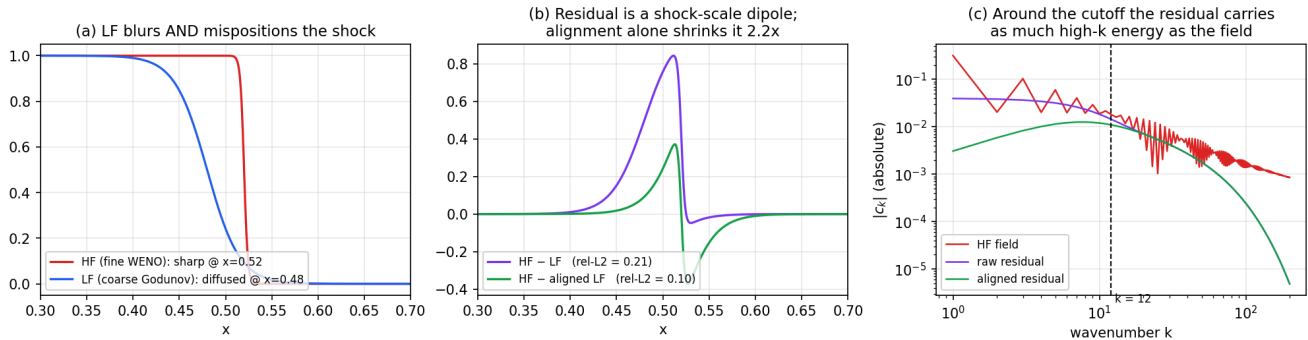


Figure 2 — When the cheap solver both blurs and shifts a shockwave, "HF minus LF" is not a small correction — it is a dipole (a positive bump right next to a negative bump) that is sharper and larger than the original field. Aligning the two shockwaves first, then subtracting, gives a much smaller and smoother correction target.

A large chunk of the model zoo predicts $\text{HF} = \text{LF-based baseline} + \text{a learned correction } (\delta)$ — the same idea as gradient boosting, where a second model cleans up the first model's residual errors. This assumes the residual is *small and smooth*, which is true when LF is simply a noisier or blurrier version of HF in the same place.

On the shockwave dataset that assumption is false in a specific, provable way. The cheap solver runs on a grid that is roughly four times coarser, using a lower-order numerical method (first-order Godunov vs. a high-order WENO scheme for the expensive version). That doesn't just blur the shockwave — it also **moves** it, typically by a few pixels. So "HF minus LF" isn't a gentle correction; it's a sharp positive bump where the true shockwave is, right next to a sharp negative bump where the LF solver incorrectly put it. A model has to learn to *cancel one shockwave and draw a new one* — which is arguably harder than learning to draw the shockwave from scratch, since it now has to do that *plus* undo a spurious feature.

The report's toy experiment shows the fix directly: if you first shift the LF shockwave onto the HF shockwave's actual position (an "alignment" or "registration" step), the leftover correction shrinks by roughly half and loses its sharp dipole shape — it becomes something much closer to smooth, small-amplitude noise, which is exactly what a correction model is good at learning.

If you know intro ML, think: gradient boosting's whole premise is "fit the residual, and the residual is small." This is the equivalent of discovering that your first model's residual is dominated by a systematic few-pixel misalignment rather than random noise — in that situation, boosting harder doesn't help; you need to fix the alignment first, the same way image registration is a standard preprocessing step before comparing two photos of the same scene taken from slightly different angles.

0.3 MSE and relative-L2 both structurally prefer blur

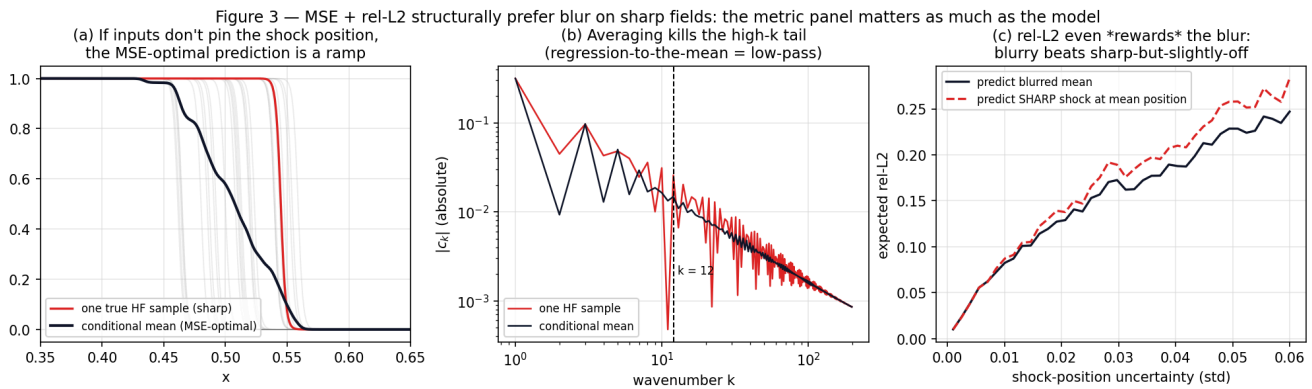


Figure 3 — If a model is uncertain whether a shockwave sits at position A or position B, the mean-squared-error-optimal prediction is a smooth ramp between them — and this blurred, hedged answer actually scores a lower (better) relative-L2 error than a sharp shockwave placed slightly off. The loss and the scoring metric are both, structurally, on the side of blur.

Every family in the zoo trains with MSE (a couple add a relative-L2 term), and every leaderboard number — per-dataset relative-L2, the pairwise-comparison Elo rating, the geometric-mean composite score used to pick an overall winner — is built from the same squared-distance idea.

Here is the subtle part. Suppose the inputs a model gets don't fully pin down exactly where a shockwave will land — maybe two training examples with very similar conditions had shockwaves in slightly different places. The mathematically optimal way to minimize *expected* squared error, when you're uncertain between two possible sharp answers, is to output their **average** — which for two shifted shockwaves is a smooth ramp, not either sharp answer. That smooth ramp is not a bug or an undertrained model; it is what MSE training is designed to produce whenever there's positional uncertainty.

It gets worse under the scoring metric. A blurred ramp, compared pixel-by-pixel against the true sharp shockwave, actually posts a *better* (lower) relative-L2 score than a perfectly sharp shockwave that's just a few pixels off — because being sharp-but-wrong is penalized twice as hard (wrong at the true location, and wrong at your own location) while a blur is only ever a little bit wrong everywhere. So a model that honestly commits to a confident, sharp answer can be scored worse than one that hedges and blurs — the exact opposite of what you'd want a benchmark to reward. This is why the report recommends adding new evaluation metrics designed specifically to catch this (per-frequency-band error, and a "how far off is the edge, separately from how sharp is it" decomposition) rather than trusting relative-L2 alone on these datasets.

If you know intro ML, think: this is the regression cousin of a well-known classification fact — a softmax classifier can express "50/50 unsure between class A and class B" without ever producing a blended, nonsensical output, because the two classes live in separate output slots. A pixel-regression model doesn't have separate slots for "shockwave at position A" and "shockwave at position B" — it has one pixel value — so its only way to express uncertainty about *position* is to blur across positions. This is exactly why generative models (which can sample one sharp, specific answer rather than average over all of them) are the standard fix for this failure mode in other fields — see Part 3.

0.4 What this predicts for the current leaderboard

A full code-level audit of every family in the zoo (same audit behind the companion model-explanations page) sorts the 30 families into seven underlying mechanisms. Two structural facts stand out. First, every FNO-based family caps its frequency budget at 12 per axis (16 or 20 for two families) — a small fraction of what the grid resolution would allow — and every resizing step between grids uses simple bilinear interpolation with no special care to avoid blurring further. Second, and less obvious: **the current winning families never look at the LF field when making a prediction.** Their multi-fidelity content lives entirely in the training schedule (pretrain on LF, fine-tune on HF); at prediction time they only see the handful of scenario numbers. That is enough when those numbers fully determine a smooth solution. It is not enough when the solution has a sharp feature whose exact position can't be inferred from a handful of numbers alone — that position is exactly what a look at the (blurry, roughly-placed) LF image would tell you, if the model were built to use it correctly.

Family group	Looks at LF image when predicting?	Why it struggles on sharp fields	Expectation on the new datasets
Pretrain → fine-tune transfer families	No — LF only shapes training	Frequency budget + blur-preferring loss; has to guess edge position from scenario numbers alone	Degrades, but likely stays the best <i>within</i> the current zoo
Physics-informed transfer variant	No (uses LF only as extra training signal, and only on one dataset type)	Same limits, plus its physics-based loss term doesn't apply to these datasets	About the same as plain transfer here
FIRE (uncertainty-aware) family, 11 variants	Yes, but only as <code>mean + uncertainty summary</code> , not the raw field	The mean baseline inherits the misplaced shockwave; correcting it hits the Wall-2 dipole problem through only 12 frequencies	Middling — better than pure "predict from scratch" families, worse than transfer
Coregionalization family	No	Its conditioning is global (whole-image rescaling), with no mechanism to nudge a local feature's position	Weak
Additive/residual "stack" families	Only a model-generated LF, not the real one	Textbook "predict the correction" pattern — exactly Wall 2	Weak
Attention-based families (Transolver, v9)	Yes — genuine LF tokens	No hard frequency cutoff, and attention <i>can</i> in principle move information around — but all of it is squeezed through a small number of summary tokens	The interesting dark horse — currently near the bottom on smooth data, might close the gap here
DINO-style families	Only a single global summary vector	A single vector has no spatial detail to correct a specific misplaced pixel	Weak

The bottom line: expect the leaderboard to roughly hold its current order on the smooth control dataset, and to invert (or come close to it) on the shockwave and interface datasets — unless the zoo adopts some of the fixes in Parts 1–4.

Part 1 — What the broader neural-network literature already knows about sharp signals

The core phenomenon has a name: spectral bias. Neural networks, in general, learn low-frequency (broad, smooth) patterns first and high-frequency (fine, sharp) patterns last or not at all — this has been shown empirically, explained theoretically, and specifically confirmed for FNOs (where the effect is compounded by the hard mode-truncation described above: the model literally cannot represent frequencies above its cutoff except through a small side-path, so it's not just "learns them last," it's "can never learn them at all" beyond that side-path's capacity).

Independent benchmarks confirm the failure mode on exactly this kind of data. The Well benchmark ran an FNO on multi-shockwave 2-D flow problems — the same physics family as this project's new Euler dataset — and measured the FNO about 2.7× worse than the best convolutional model tested (a ConvNeXt-U-Net; an ordinary U-Net also beat the FNO by a wide margin). APEBench found the same pattern more generally: spectral models like FNO win when a system's energy concentrates into a few dominant frequencies, and lose once the frequency spectrum is fully populated (which sharp, turbulent, or shock-laden fields tend to be). A third analysis (behind PDE-Refiner, a well-known operator-learning paper) explains the training-side mechanism simply: under MSE, the error contributed by each frequency is roughly proportional to that frequency's *amplitude squared* — and sharp features put a lot of their energy in low-amplitude high-frequency components, which MSE training essentially ignores because they contribute so little to the total loss. This is true independent of architecture; it's a property of the loss function itself.

The fixes the literature has found, organized from cheapest to most involved:

- **Free or nearly free (just change the loss or training schedule).** Train a small second model on the first model's leftover error (its spectrum is much flatter, so it's easier to fit) — this alone has been shown to cut error by roughly half, concentrated exactly in the frequencies the first model missed. Gradually admit higher frequencies during training rather than all at once. Weight the loss more heavily near edges, so those pixels aren't statistically drowned out. Add a loss term computed directly on frequency bands rather than raw pixels.
- **Cheap architecture changes.** Add a small ordinary-convolution branch alongside the Fourier branch — consistently the single most validated fix in the literature, motivated by exactly this kind of sharp-feature problem. Raise the number of retained frequencies (a smarter parameterization can do this without a parameter-count explosion). Swap the standard ReLU-style activation for one that can represent non-smooth functions more efficiently.
- **Bigger architecture changes.** Replace the Fourier layer with an "alias-free" convolutional operator, shown to beat FNO specifically on shockwave and discontinuous-transport benchmarks. Or use wavelets instead of a plain Fourier transform — wavelets localize a feature in both space *and* frequency simultaneously, which a plain Fourier transform cannot do. Or use an attention/transformer-style operator, which has been demonstrated on transonic-airfoil shockwave problems directly.
- **Generative refinement.** Predict the smooth, overall structure with a normal deterministic model, then run a few denoising (diffusion-style) steps to add back plausible high-frequency detail — this

has specifically been tested on the same chaotic-but-smooth control system used here (Kuramoto–Sivashinsky). A more recent variant trains the whole operator adversarially (in the same spirit as a GAN) and reports roughly a 15× reduction in high-frequency spectral error on a related super-resolution task, at the same prediction-time cost as a normal deterministic model.

- **Purpose-built shockwave models already exist** (for the single-fidelity case, i.e., without the multi-fidelity fusion problem this project adds) — operator-learning architectures specifically designed for extreme pressure/density jumps, and learned "shock sensors" that can be used as extra input features to any of the above.

The prediction, stated plainly: the FNO-vs-local-model ranking that this project would see on smooth data is expected to *flip* on data with a fully populated frequency spectrum — which is exactly the situation for the shockwave and interface datasets — unless the zoo adopts one or more of the fixes above.

Part 2 — What the multi-fidelity literature already knows about misaligned cheap/expensive pairs

2.1 A 25-year-old formula, and exactly when it breaks

The classical multi-fidelity recipe from statistics (published in 2000, still widely used) is almost exactly $HF \approx (\text{a scale factor}) \times LF + \text{a smooth correction}$, with the correction modeled as a smooth random process. That formula's built-in assumption — the correction is smooth — is precisely what a mispositioned shockwave violates.

A well-known empirical rule of thumb puts a number on this: this kind of fusion needs the cheap and expensive versions to be correlated with each other at roughly $r^2 \geq 0.9$ (think of r^2 the way you would in a regression class: the fraction of variance the cheap version explains about the expensive one) before it beats just using expensive data alone. A separate, more theoretical result — about the best possible use of a mix of cheap and expensive samples, not tied to any one model — proves that even in the best case, you're left with a fraction $1 - \rho^2$ of the variance unexplained, where ρ is the LF/HF correlation. In plain terms: **the less LF and HF agree, the less benefit any fusion method can extract from LF, no matter how clever it is** — and shockwaves (or chaos, for the KS dataset) are exactly what drives that correlation down.

The two classical escape routes both point the same direction as this report's main recommendation: feed the LF *value* into the HF model as a flexible, learnable input rather than assuming a fixed additive relationship, or explicitly learn a *warping* of the input space before combining the two fidelities — an early precursor of "align first."

2.2 Four different fields independently invented the same fix

This is one of the more striking parts of the literature sweep: at least four unrelated research communities hit the exact same wall (misaligned sharp features break naive comparison/correction) and converged on

the exact same two-step fix — **first estimate a smooth shift/warp that lines up the two versions, then correct the remaining amplitude difference** — decades apart and without citing each other.

- **Weather forecasting**, roughly 30 years ago: a forecast that puts a storm in the right shape but the wrong place used to be scored as "completely wrong" by naive pixel comparison — exactly the shockwave problem in this report. The field's fix, still standard practice, is to first solve for a smooth displacement field that morphs the forecast onto the observation, and only then score the amplitude that's left over.
- **Seismic imaging**: comparing two wavefields with a standard squared-error-style comparison creates spurious "worse-is-further-away" traps whenever the two are shifted — the seismic-imaging fix uses a comparison method (optimal transport) that is provably well-behaved under pure shifts, unlike squared error.
- **Statistics**: a general calibration technique explicitly splits a mismatched pair of curves into an "amplitude" part and a "warping/phase" part, and models the two separately rather than jointly.
- **Aerodynamics** (the industrial home of shockwave-heavy engineering): a technique called shape-preserving response prediction cheaply corrects a pressure-distribution curve by tracking and translating its *characteristic points* (where the shock sits, where the suction peak is) from the cheap prediction onto the expensive one, rather than doing a pixel-wise correction; a related technique, manifold-alignment reduced-order modeling, compresses both the cheap and expensive fields into a shared low-dimensional latent space and fuses them there with a closed-form linear alignment step (Procrustes analysis).

If you know intro ML, think: this is the same idea as image registration or optical flow, applied as a preprocessing step before you diff two images — instead of computing "pixel-wise difference between misaligned photo A and photo B" (which is dominated by edges moving past each other), you first estimate how to warp A onto B, and only then look at what's left over. No published multi-fidelity *operator-learning* method does this yet — it's the report's flagship proposal (P3, below).

Why the aerodynamics precedent doesn't already count as "someone built this": both aerodynamics techniques above are per-query, hand-engineered corrections, not general offline-trained neural networks. Shape-preserving response prediction needs a fresh, real expensive (HF) simulation at nearly every point it's applied to — it's a local correction step inside an iterative design-optimization loop, not a model trained once and then asked to generalize to brand-new scenarios with zero further expensive evaluations, the way this project's models must. It also relies on hand-identified landmarks (someone has to define and detect "the shock location" and "the suction peak" on a pressure curve), not an automatically-learned displacement over an arbitrary field. Manifold-alignment ROMs fuse the two fidelities with a closed-form linear alignment step fit once per problem class, not a differentiable neural network that learns a continuous displacement field end-to-end. What proposal P3 targets is the version neither of these is: a neural operator that, once trained, predicts the aligned-and-corrected HF field from *only* the LF field and a handful of scenario numbers — no further expensive evaluations, no hand-identified landmarks. That's genuinely useful context, not a contradiction: it means P3 is automating and generalizing an idea aerodynamics engineers have already hand-validated works, rather than inventing one from nothing.

2.3 A theorem, and the gap it points at

A 2023 theoretical result proves something specific and relevant: operator-learning architectures that use *linear reconstruction* — meaning, roughly, that the output is built as a fixed linear combination of learned basis functions, which describes architectures such as DeepONet and PCA-Net — are provably inefficient at representing discontinuous solutions, no matter how they're trained. The paper's own proposed fix is an architecture that explicitly learns input-dependent *shifts* — again, an alignment mechanism, arrived at from pure theory this time rather than practical experience.

A separate, useful structural fact from the model-order-reduction literature: **the displacement/warp field that aligns two versions of a discontinuous signal is itself smooth**, even when the signal it's warping is not. That matters a lot for this project, because it means "predict the warp" is a much friendlier learning problem — smooth targets are exactly what neural networks are good at — than "predict the sharp field directly."

Existing multi-fidelity work for PDE operators, by contrast, is essentially all either plain pretrain-then-finetune transfer learning (the same family this project's zoo already has) or a fixed linear-plus-nonlinear correction formula. None of it does the align-then-correct trick yet — which is exactly the gap proposal P3 aims to fill.

2.4 The smooth control dataset (Kuramoto–Sivashinsky) has its own, different theory

The KS dataset isn't sharp — it's smooth but *chaotic*, meaning small differences compound over time. There's a separate, proven result here: once a cheap and an expensive simulation of a chaotic system have run long enough to decorrelate (chaos amplifies tiny initial differences exponentially), there is a hard accuracy limit on any pointwise correction method, no matter how good — beyond that point, only statistical properties (average energy spectrum, long-run statistics) remain predictable, not the exact pointwise field. The practical takeaway for this project: on the KS dataset, whether multi-fidelity fusion helps *at all* likely depends on how long the simulation has run relative to that decorrelation time — worth measuring directly before concluding any particular model is bad.

2.5 The negative results, all pointing the same way

Four separate, independently-derived formal results all say the same thing from different angles: the classical "correlation must be high" threshold, the "leftover variance never fully vanishes" bound, the "linear-reconstruction architectures are inefficient on discontinuities" theorem, and the "chaotic decorrelation caps pointwise accuracy" limit. All four disadvantage the simple $HF = LF + \text{correction}$ pattern that dominates the current zoo, and all four point toward the same family of fixes: align/warp before correcting, feed LF in as a flexible input rather than an additive baseline, use features that are aware of discontinuities, and (for the chaotic case) match statistics rather than exact pixels.

Part 3 — Borrowed ideas: other fields solved "cheap blurry → expensive sharp" first

Half a dozen fields outside of PDE modeling have spent decades on essentially this exact problem. One thing to keep in mind throughout: because this project is scored on relative-L2 (a squared-error-style metric), a blurry *average* answer is mathematically optimal *unless* the fix also solves a position/phase error (Wall 2) or the evaluation itself is changed (Wall 3, Part 0.3) — so the ideas below that pay off most are the ones that fix *where* things are, sharpen in a deterministic way, or use a generative model's *average of several samples* rather than a single random sample.

Cosmology and turbulence simulation — "hallucinate" texture, then check it against statistics, not pixels. Both fields gave up on recovering exact pixel-level fine detail from a coarse simulation, because that detail genuinely isn't determined by the coarse version — instead, a generative model (GAN or diffusion) samples *statistically plausible* fine detail, and the result is checked against summary statistics (energy spectra, typical cluster sizes) rather than pixel-by-pixel accuracy. Pure squared-error training was shown early on to oversmooth in exactly this setting; adversarial training fixed the oversmoothing and recovered the correct fine-scale statistics. *What could transplant here:* reweight the training loss by frequency band according to which bands the model is currently getting most wrong; log the frequency spectrum of predictions vs. ground truth as a standard diagnostic regardless of which model is used, since it directly answers "how far from a perfect model are we, and at which scales."

Computer-vision super-resolution — the blur problem is well-documented, with several fixes at different costs. A landmark paper first precisely documented "regression to the mean" blur in image super-resolution. The cheapest fix is a different loss (one that behaves more like a median than a mean under positional uncertainty, so it stays sharper) or dynamically upweighting whichever frequency bands the model is currently wrong about. A pricier fix changes the architecture to predict corrections at multiple resolution bands with explicit supervision at each level, or to decode any arbitrary output coordinate from local features (relevant here since this project's cheap and expensive fidelities live on different-resolution grids, and naive resizing between them injects blur before training even starts). The most expensive fix is full generative super-resolution — only worth it if the evaluation metric is changed to reward realism, not squared error.

Audio bandwidth extension — never re-predict what the cheap signal already got right. Systems that extend a telephone-quality (narrowband) audio signal to full bandwidth don't try to regenerate the whole signal — they copy the already-correct low frequencies verbatim and generate only the missing high frequencies. *The transplant:* a model that hard-copies the (lightly corrected) low frequencies from the cheap solution and only predicts the frequencies above some cutoff — with that cutoff estimated per dataset from how well cheap and expensive agree at each frequency. This directly encodes "trust the cheap solver where it's accurate, and only invent detail where it isn't."

Seismic imaging — the misalignment problem has a 30-year-old industrial fix. Comparing two seismic wave recordings with ordinary squared error creates the exact same trap as Wall 2: get the timing of a wave slightly wrong, and squared error punishes you twice (once for being early, once for missing where the true wave was) — a phenomenon called "cycle skipping" that traps optimization in bad local solutions. The two standard industrial fixes: fit heavily-smoothed (low-pass) data first, then

gradually let in finer detail as training progresses; or replace squared-error comparison with an optimal-transport-based comparison, which is mathematically well-behaved specifically under pure shifts. *The transplant*: both a training-schedule trick (gradually raise the amount of fine detail the target is allowed to contain) and a loss-function trick (an auxiliary term that penalizes *how far* a feature needs to move, turning "shockwave 3 pixels left" into a small, well-behaved penalty instead of a squared-error trap).

Classical numerical analysis — four decades-old theorems that map almost directly onto architectures. A multigrid technique for combining coarse and fine grid solutions defines "what the coarse solver got wrong" as a precise mathematical correction term (not a fuzzy image difference) — this is essentially what the cheap/expensive relationship in this project is, if the fusion mechanism is built to respect it. A shock-capturing numerical method (WENO) refuses to interpolate straight across a discontinuity by design, using a nonlinear, edge-aware weighting scheme instead of the naive bilinear averaging every family in this zoo currently uses. A classical result about reconstructing a function from truncated frequency data proves that if you *also* know where the discontinuities are, you can recover very high (in nice cases, exponential) accuracy in between them — directly relevant since FNO's core output is exactly a truncated set of frequencies. And a family of specialized frequency-like transforms (curvelets/shearlets) is mathematically proven near-optimal specifically for images made of smooth regions separated by curved edges — precisely the shape of a shockwave or interface field.

Graphics and face restoration — treat missing detail as a lookup problem, not a regression problem. Graphics has stored fine surface detail separately from broad shape since the 1980s, and photorealistic face restoration under severe damage works not by regressing pixel values directly but by *classifying* the damaged input against a small learned dictionary of plausible, high-quality structures — a much stronger prior than open-ended regression when data is scarce and the missing structure is highly stereotyped. *The transplant*: shockwave cross-sections and phase-interface profiles are, in fact, extremely stereotyped (a Cahn–Hilliard interface profile is essentially always the same universal S-shaped curve) — a small learned dictionary ("codebook") of typical high-fidelity detail patches, looked up from cheap-field features, is a training-data-efficient alternative to regressing raw pixels from scratch.

Physics itself — coarse-graining destroys information, and what's destroyed is not arbitrary. A physics result says that "undoing" a coarse-graining step is *fundamentally* a statistical, not deterministic, operation — matching the cosmology lesson above. A separate, sharper mathematical fact from homogenization theory says a coarse solver isn't computing a blurred version of the true field — it's solving a genuinely *different* (simplified) equation, and the missing fine-scale detail is mathematically tied to the *gradient* of the coarse solution. *The transplant*: feed the gradient (and curvature) of the cheap field in as explicit extra input channels, and predict corrections in a coordinate frame that's locally rotated to align with that gradient — this is a way of getting one model to handle shockwaves at any orientation without needing separate training examples for each orientation.

Computer vision and molecular ML — bake in the symmetries the physics already has. Two fields well outside PDE modeling have spent the last decade building exactly this kind of prior: rotation/reflection-equivariant CNNs in computer vision, and equivariant networks for molecules and particle systems in physics-and-chemistry ML. Both rest on the same observation. A single PDE solution — one specific shockwave, one specific interface — is not symmetric, because the boundary and initial conditions that produced it usually aren't symmetric either. But the *solution operator* — the underlying rule mapping "problem setup" to "solution," which is what a neural network here is actually trying to learn

— inherits whatever symmetries the governing equation has, as long as the domain and boundary conditions also respect that symmetry and the solution is unique. If nothing in the equation prefers one direction or position over another (true for the Laplacian everywhere, true for most PDEs away from any fixed wall or source), then rotating the input and rotating the output are the same operation mathematically, and a network that doesn't already know this has to spend scarce training data rediscovering it from scratch, separately, at every position and orientation. There is a genuine extra subtlety for a vector field like Euler's velocity: rotating the domain has to rotate the velocity's *direction* too, not just move its pixels around, so an equivariant architecture for Euler needs to treat some of its channels as vector-valued and some (density, energy) as plain numbers — a real extension over the image-style equivariant layers built for vision, not a drop-in. And it only helps on datasets where the domain and boundary conditions are themselves symmetric — a fixed wall or a heat source at a specific location genuinely needs the network to know where it is, so this is a per-dataset judgment call, not a blanket fix.

If you know intro ML, think: this is the actual reason CNNs beat plain fully-connected networks on images in the first place. A convolution is translation-equivariant by construction — a cat in the top-left of a photo and a cat in the bottom-right get recognized by literally the same learned filter, instead of the network having to independently learn "cat" from separate training examples at every possible pixel location. Rotation/reflection equivariance is the identical idea one level up — the architectural version of the "randomly rotate and flip the training images" data-augmentation trick every intro computer-vision course teaches, except built into the network's structure instead of approximated by resampling the data.

Why this is worth taking seriously specifically here, and not just in general: every dataset in this project is bottlenecked by the same thing — plenty of cheap LF data, very little expensive HF data. Each symmetry a network doesn't have to learn from data is, in effect, a free multiplier on the number of HF training examples it effectively has: one labeled shockwave also teaches the network about every rotated, reflected, and (where applicable) translated copy of that shockwave, not only that one exact instance. That is exactly the kind of leverage a data-starved project should be looking for.

One finding is worth flagging directly, because it shows this is not a hypothetical: **this project's own shared FNO backbone code currently does the opposite, on purpose.** The shared `fire_core.py` module and the `mf_fno_pinn_transfer` family both explicitly build a grid of absolute (x, y) coordinates and feed it into the network as an input channel, which breaks translation equivariance by design — every family built on that code is told exactly where it is on the grid, rather than being structurally forced to treat a feature the same way regardless of position. That is presumably there because some datasets need it (fixed boundaries), but on datasets where the domain is periodic or the physics has no preferred location — true for a good chunk of the *original* smooth suite too, not only SURF — this is currently spending model capacity relearning something the architecture could give away for free.

The transplant: the cheapest version costs almost nothing — add rotated and mirror-flipped copies of every training example (the exact 90°/180°/270°-rotation-plus-mirror symmetry of a square grid, called the D4 group, needs no interpolation since grid points map exactly onto other grid points) to any existing family's training loop, and separately test whether removing the raw absolute-position input channel hurts or helps on the datasets whose domain and boundary conditions are actually symmetric. A dedicated equivariant-backbone family that bakes D4 rotation and reflection in architecturally, rather than

approximating it via data augmentation, is the more ambitious version — and would need genuine vector-aware handling for Euler's velocity field specifically.

A grab bag of smaller, concrete ideas: separating a signal into "where is it" (phase) and "how strong is it" (amplitude) and correcting each separately, an idea from video frame interpolation, is architecturally the same move as the alignment idea above; importance-sampling training patches by how much error they currently produce (a technique from image demosaicing) is close to free, since on sharp datasets a small fraction of pixels typically carries most of the total error; and there are already several 2025–2026 papers combining more than one of these ideas at once — evidence that this direction is both timely and increasingly competitive.

Part 4 — The nine proposed model families, explained plainly

Every proposal below fits the repo's existing contract: lives at `models/<family>/`, has a manifest and a fixed-CLI smoke-eval script, uses the shared data-loading code, supports checkpoint resume, and finishes a smoke run in under 30 minutes on one GPU. The ranking below balances expected benefit against how risky and how much work each one is.

Tier 1 — cheap experiments that isolate the problem first

P1 — ``mf_fno_transfer_local``: give the winning recipe more frequencies and a local-detail path.

What changes: take the current best-performing recipe exactly as-is (same pretrain-then-finetune schedule, same FiLM conditioning) and change only the backbone block — raise the retained frequency count from 12 to 32, using a parameter-efficient reformulation so this doesn't blow up the parameter count, and add a small parallel ordinary-convolution branch next to the Fourier branch in every block.

Why: this directly removes the Wall-1 frequency ceiling and adds exactly the "local path" fix the literature (and The Well's benchmark numbers) says matters most.

If you know intro ML, think: this is the neural-operator equivalent of adding a skip-connection-style local branch to a global-pooling model — like adding a small CNN branch alongside a vision transformer's global attention, so the model has both a "big picture" path and a "fine detail" path.

Risk / cost: low risk, a few days. Every ingredient here is well-established in the literature, not speculative.

P2 — ``mf_fno_transfer_sharploss``: same architecture, just a sharper loss function.

What changes: identical model to the current best recipe — change only the training loss (a loss that behaves more like a median than a mean under positional uncertainty; extra weight on pixels near strong LF gradients, i.e., likely edges; a loss term computed on frequency bands; and a training schedule that gradually admits sharper detail into the target over the first third of training).

Why: this isolates Wall 3 from Wall 1 — since P1 changes only the architecture and P2 changes only the loss, comparing their results cleanly answers "is the bottleneck the model's frequency budget, or the objective it's trained on?"

Risk / cost: minimal risk (it's a loss-only change), one to two days.

Tier 2 — the genuinely new multi-fidelity mechanisms

P3 — ``mf_warp_correct``: align first, then correct (the flagship proposal).

What changes: instead of predicting ``HF = LF + correction`` directly, first predict a smooth 2-D displacement field that describes how to shift/warp the LF image to line its shockwave up with where the HF shockwave actually is, apply that warp to the LF image, and only then add a learned correction on top. The displacement head starts at "do nothing" so the model can gracefully fall back to plain correction wherever alignment isn't needed. As an optional warm start, the displacement targets can be pre-computed directly from paired training data using standard optical-flow techniques, which makes the otherwise-tricky joint optimization much easier to get off the ground.

Why: this is the exact mechanism that four independent fields (Part 2.2) converged on, and no published multi-fidelity operator-learning method does it yet — this is also the proposal most likely to be genuinely novel and publishable, not just an incremental engineering fix.

If you know intro ML, think: this is image registration (or optical flow) as an explicit, trainable preprocessing step inside the model, rather than assuming two images are already pixel-aligned before you diff them.

Risk / cost: medium risk (jointly training a warp and a correction together can be finicky, though the optical-flow warm-start mitigates this), roughly a week.

P4 — ``mf_band_split``: never re-predict what the cheap solver already got right.

What changes: hard-constrain the model's output so its low frequencies come directly (with a small learned correction) from the cheap solution, and have the network predict only the frequencies above a cutoff — with that cutoff estimated per dataset from where the cheap and expensive solutions stop agreeing well.

Why: the direct transplant of the audio-bandwidth-extension idea (Part 3); concentrates the scarce expensive training data on the part of the signal that's actually missing, instead of re-learning parts the cheap solver already solved correctly.

Risk / cost: medium — this proposal explicitly assumes the low frequencies are already correctly *positioned*, which Wall 2 says isn't quite true; it pairs naturally with P3, which fixes exactly that assumption. Roughly three to four days.

P5 — ``mf_specb_residual``: a second, smaller model that mops up whatever frequencies the first model missed.

What changes: freeze the best transfer model from P1/P2, then train a small second model on its leftover error — whose frequency spectrum is much flatter (easier to fit) than the original field's spectrum.

Why: purpose-built for "the first model nailed the smooth part; what's left is specifically the sharp part" — and because the second stage is small, it fits comfortably within the compute budget.

Risk / cost: low risk, two to three days. One thing to watch: the second stage needs to see edge-aware input features (like the gradient of the cheap field), or it will simply inherit the same spectral-bias problem on a now-harder target.

Tier 3 — dataset specialists and bigger bets

P6 — `mf\_interface\_levelset`: for the phase-interface dataset specifically, predict shape parameters instead of pixels.

What changes: rather than predicting the sharp field directly, predict a smooth "distance-to-interface" map plus two smooth amplitude values, then combine them with a known, fixed mathematical formula (the same S-shaped curve that phase-separation interfaces always follow) to produce the final sharp field.

Why: the network only ever has to predict smooth quantities — the sharpness is added afterward by the fixed formula, so the frequency-ceiling problem never applies at all.

Risk / cost: medium risk (handling topology changes, and needing an extra preprocessing step to extract the interface shape from training data), about a week; potentially a very high ceiling specifically on the interface dataset.

P7 — `mf\_gen\_refiner`: add a few generative "sharpening" steps on top of a normal prediction.

What changes: run the current best deterministic model, then apply a handful of diffusion-style denoising steps conditioned on that prediction to restore plausible fine detail; average several sampled outputs at evaluation time so the score doesn't get worse due to sample-to-sample randomness.

Why: this is the only proposal that can *honestly* add back detail that genuinely isn't recoverable deterministically from the cheap input — the same logic cosmology and turbulence super-resolution rely on — and it would win big on a metric that scores spectral realism, even in cases where it ties on relative-L2.

Risk / cost: medium-to-high risk given the tight compute budget (though a published recipe suggests a cheap enough version — roughly 20 training epochs and 20 sampling steps — fits), about a week.

P8 — `mf\_patch\_codebook`: a small learned dictionary of typical sharp-detail patches (the creative bet).

What changes: train a small discrete "codebook" of typical high-fidelity detail patches found near strong cheap-field gradients, plus a lightweight model that looks up the right codebook entries from cheap-field features, and blend the looked-up detail into the baseline prediction through a learned confidence gate.

Why: shockwave and interface cross-sections are about as stereotyped as facial features are in face-restoration work, where exactly this "classify against a small dictionary" trick strongly outperforms open-ended regression under scarce training data — and each expensive training example donates roughly a thousand small patches, which sidesteps the usual "not enough expensive data" constraint.

Risk / cost: high risk (this is a genuinely novel architecture, and the confidence gate has to be trustworthy), about a week; a genuinely new kind of family if it works.

P9 — `mf\_fas\_multigrid`: treat the cheap solution as a numerical correction term, not an image.

What changes: unroll two or three steps of a classical multigrid-style correction cycle — compute what the current estimate gets wrong relative to the actual cheap solve (in the precise numerical sense, not an image-difference sense), pass that through a small learned model, and upsample using an edge-aware (not naive-averaging) interpolation scheme.

Why: arguably the deepest use of the cheap/expensive relationship on this list, directly borrowing 40-year-old multigrid theory built for exactly this kind of coarse/fine relationship.

Risk / cost: medium-to-high risk (real engineering complexity in handling different dataset grid geometries).

A few cheap wins that apply no matter which proposal gets built

1. **Feed edge-aware input channels everywhere:** the gradient of the cheap field, and a couple of related "how rough is it here" indicators — cheap to compute, and physics theory (Part 3, homogenization) directly says the missing fine detail is tied to exactly this gradient.
2. **Oversample the hard pixels:** on sharp datasets, a small fraction of pixels typically accounts for most of the total error — training more on those specific crops is nearly free and reuses the existing compute budget more effectively.
3. **Build a proper evaluation panel** (not just relative-L2): per-frequency-band error, a "how far off is the edge, separately from how sharp is it" decomposition, a sharpness-preservation ratio, and the frequency-truncation floor from Figure 1 as the honest "best a 12-mode model could ever do" reference line.
4. **Measure before modeling:** compute, per dataset, how well cheap and expensive solutions actually correlate, how well their frequencies agree at each scale, and (for the chaotic control dataset) how much time has passed relative to the point where cheap and expensive trajectories decorrelate. These three numbers predict where multi-fidelity fusion can help *at all*, and will explain most of the leaderboard shifts before a single new model is trained.
5. **Add rotation/reflection data augmentation where the domain allows it** (Part 3, symmetries): nearly free to bolt onto any existing family's training loop on datasets with a periodic or otherwise symmetric domain, and a direct multiplier on the effective number of scarce HF training examples.

Suggested build order

P1 and P2 first (days, low risk, and together they cleanly separate "is it the architecture or the loss?"). Then P3 and P5 (the genuinely new multi-fidelity mechanisms). Then P4 and P6 for dataset-specific gains. P7–P9 last, as bigger research bets, once results from the first round are in.

Part 5 — Answers to the original open questions

"How well are the models doing, and how do I visualize best-model-vs-perfect-model?" Three visualizations, from simplest to most informative: (a) plot each model's error against the best score a 12-mode model could ever achieve (the floor from Figure 1) — on smooth data, the whole gap to zero is real model error; on sharp data, much of that gap is the un-fixable frequency floor, not the model's fault; (b) plot error broken down by frequency band, which shows exactly which scales a model is winning or losing at (a perfect model would be a flat zero line across all bands); (c) separately report how far off a shockwave's *position* is versus how much *amplitude* error is left once position is corrected — this splits "sharp but slightly misplaced" from "genuinely blurry," two very different failure modes that a single relative-L2 number blends together.

"How do the models work, and how would I build one for high-frequency data?" The per-family walkthroughs live in the companion page ([model_explanations.html](#) / its beginner edition); the high-frequency-specific answer is this document and the technical report it summarizes.

"What about using reinforcement learning — the Elo ranking reminded me of RL?" The resemblance is only skin-deep. Elo, here, is just a way of *summarizing* which model wins more often across datasets — it's an evaluation tool, not a reward signal any agent is being optimized against. As a way to actually train the prediction models, RL would replace exact, low-variance gradients (which is what training a dense pixel-by-pixel prediction with backpropagation gives you) with much noisier, higher-variance policy-gradient estimates — for this kind of problem, ordinary differentiable training is strictly better and cheaper. Where RL genuinely does belong in multi-fidelity work is a different problem entirely: *deciding which fidelity to query, where, and when*, under a limited budget — but this project's datasets are fixed in advance, so that decision problem doesn't currently exist here. A reasonable middle ground — using something RL-flavored to decide which training examples to emphasize — is likely matched just as well by a simpler, fully differentiable alternative: weighting the loss by the model's own current uncertainty estimate, which several families already compute as a byproduct.

Glossary — terms specific to this report

Shock / discontinuity. A place where a physical field jumps almost instantly from one value to another — a shockwave in compressible flow, or the boundary between two phases in Cahn–Hilliard. The defining property, for this report, is that its frequency content doesn't decay quickly, unlike a smooth field's.

Fourier mode / frequency. One sine-wave "ingredient" of an image's Fourier decomposition. Low modes describe broad shapes; high modes describe fine edges and detail. "Keeping 12 modes" means keeping only the 12 lowest-frequency ingredients per axis and discarding the rest.

Spectral bias. The general tendency of neural networks to learn low (smooth) frequencies faster and more easily than high (sharp) frequencies. Compounded for FNOs, where high frequencies above the mode cutoff can't be represented at all, not just learned slowly.

Spectral wall / truncation floor. The hard ceiling on accuracy that results from throwing away high frequencies before training even starts — no amount of training removes this floor; only keeping more frequencies (or changing architecture) does.

Residual / correction (δ). The gap between a baseline prediction and the true answer, $\text{true} = \text{baseline} + \delta$. Learning δ instead of the whole answer is the same idea as gradient boosting; it only works well when δ is small and smooth.

Misalignment / dipole. What happens to a residual when the baseline doesn't just blur a sharp feature but also shifts its position — the residual becomes a sharp positive/negative pair (a "dipole") rather than small smooth noise.

Warp / registration / optical flow. A smooth mapping that describes how to shift or deform one image so its features line up with another image's features — a standard preprocessing step in vision and forecasting before directly comparing two images.

Co-kriging. A classical statistical technique for combining a cheap and an expensive version of the same quantity, assuming their relationship is a scale factor plus a smooth correction.

Correlation / r^2 threshold. How much of the expensive version's variation is explained by the cheap version. Below roughly 0.9, classical multi-fidelity fusion methods stop being worth using over just using expensive data alone.

Optimal transport. A way of comparing two distributions (or images) by the minimum "work" needed to move one onto the other, rather than a pixel-by-pixel difference — well-behaved under pure shifts, unlike squared error.

GAN (generative adversarial network). A generative model trained by having a second "discriminator" network try to tell its outputs apart from real data; historically the standard fix for MSE-induced blur in image generation and super-resolution.

Diffusion / denoising model. A generative model that starts from noise (or a rough prediction) and iteratively removes noise over several steps to produce a sample; can restore plausible fine detail that a single deterministic prediction would blur away.

Multigrid / FAS (Full Approximation Scheme). A classical numerical technique for solving equations on a hierarchy of coarse-to-fine grids, where the exact mathematical relationship between a coarse and fine solution is well-defined — the numerical-analysis ancestor of "multi-fidelity fusion."

WENO (Weighted Essentially Non-Oscillatory). A numerical scheme designed specifically to interpolate near sharp features without smearing them, by nonlinearly choosing which side of a discontinuity to trust — the opposite of the naive bilinear interpolation used throughout the current model zoo.

Curvelets / shearlets. Specialized frequency-like transforms, related to wavelets, mathematically proven to be especially efficient at representing images made of smooth regions separated by curved edges.

Level set / signed-distance field. A smooth function whose zero-crossing traces out a sharp boundary — a standard way to represent a sharp interface using only smooth quantities (used in proposal P6).

VQ codebook (vector-quantized codebook). A small, learned dictionary of typical patterns; instead of generating output by open-ended regression, a model looks up (or blends) entries from this dictionary — used in proposal P8.

Homogenization / renormalization group (RG). Physics and mathematics theories describing what information is destroyed when a fine-grained system is replaced by a coarse, effective one — used in this report to argue that "undoing" a coarse simulation is fundamentally statistical, not deterministic, and that the missing detail is tied to the coarse solution's gradient.

Solution operator. The underlying rule mapping a PDE's problem setup (source term, boundary/initial conditions) to its solution field — the thing a neural operator like FNO is actually trying to learn. Individual solutions are usually not symmetric, but the solution operator inherits the governing equation's symmetries whenever the domain and boundary conditions respect them too.

Equivariance / invariance. A network is equivariant to a transformation (translation, rotation, reflection) if transforming the input transforms the output the same way, instead of the network having to relearn the same pattern separately at every position or orientation. Standard convolutions are translation-equivariant by construction — the same underlying reason CNNs beat plain fully-connected networks on images.

D4 (dihedral group of a square). The 8-element symmetry group of exact $90^\circ/180^\circ/270^\circ$ rotations plus mirror flips — the largest set of rotation/reflection symmetries a square grid supports without needing interpolation, since every transformation maps grid points exactly onto other grid points.

Full citations (arXiv IDs, journal references) for every claim above are in the companion technical report, [MF_Sharp_HighFreq_Report.pdf](#) — this document deliberately omits them to stay readable, but makes no claim that isn't sourced there. Figures reused: [report_figs_hf/fig_hf1_spectral_wall.png](#), [fig_hf2_misalignment.png](#), [fig_hf3_mean_blur.png](#). Companion pages: [MFFP_Model_Explanations_Beginner.html](#) (the model zoo, from scratch) and [MF_SOTA_vs_MFFP_Report.pdf](#) (the smooth-suite SOTA landscape).